

ARIADNE: Reconstruction of Stateful Web Attacks via Goal-Oriented State Search

Yihao Peng¹, Jingrun Zhang¹, Xinyu Mu¹, Zhuofeng He¹, Shouling Ji², *Senior Member, IEEE*, Hai Wan¹,
and Xibin Zhao¹, *Senior Member, IEEE*

Abstract—Modern web applications are inherently stateful, where exploiting critical vulnerabilities often requires a precise sequence of requests to prepare the backend state. In practice, intrusion detection systems and SIEM platforms typically trigger alerts only upon the final payload-carrying request that exhibits anomalous behavior. However, reconstructing the full attack chain for root cause analysis remains difficult because the preceding state-setting requests are often buried in massive, noisy traffic logs. We propose ARIADNE, a goal-oriented framework that automatically reconstructs prerequisite request sequences given only the terminal attack request. ARIADNE formulates reconstruction as a guided state-space search. It derives a target backend state from the terminal request and navigates the search space using a novel state-distance metric based on bipartite graph matching. To improve efficiency, ARIADNE leverages data-driven API dependencies and a state-aware parameter sourcing mechanism. Evaluated on 15 real-world vulnerabilities, ARIADNE successfully reconstructed all attack chains up to 8 steps within 20 minutes. Conversely, state-of-the-art baselines failed on all attacks requiring more than two steps, even when equipped with the necessary attack payloads.

Index Terms—API Dependency, attack chain reconstruction, root cause analysis, state-space search, stateful web services.

I. INTRODUCTION

MODERN enterprise web applications have transitioned into highly stateful architectures, where critical business logic is distributed across complex API ecosystems [1], [2]. This evolution has shifted the threat landscape toward multi-step stateful attacks. In these scenarios, an adversary does not trigger a vulnerability through a single, self-contained malicious request. Instead, they execute a sequence of “state-setting” requests that incrementally manipulate the backend database or file system, establishing the necessary preconditions for a final request to execute a malicious payload [3], [4], [5].

Received 9 April 2026; revised 31 May 2026; accepted 5 July 2026. Date of publication 8 July 2026; date of current version 1 September 2026. This work was supported by NSFC Program under Grant 6212780016. (Yihao Peng, Jingrun Zhang, Xinyu Mu, and Zhuofeng He contributed equally to this work.) (Corresponding author: Xibin Zhao.)

Yihao Peng, Jingrun Zhang, Xinyu Mu, Hai Wan, and Xibin Zhao are with the Beijing National Research Center for Information Science and Technology (BNRist), Key Laboratory for Information System Security, Ministry of Education (KLIS), School of Software, Tsinghua University, Beijing 100084, China (e-mail: xy20130630@gmail.com; jingrun-24@mails.tsinghua.edu.cn; muxu24@mails.tsinghua.edu.cn; wanhai@tsinghua.edu.cn; zxb@tsinghua.edu.cn).

Zhuofeng He is with the School of Business, Central South University, Changsha 410083, China (e-mail: zhuofeng@csu.edu.cn).

Shouling Ji is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China (e-mail: sjj@zju.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3711370

In industrial security operations, this multi-step nature creates a significant visibility gap. Security Information and Event Management (SIEM) systems [6], [7], [8], [9], [10], [11] and Intrusion Detection Systems (IDS) [12], [13], [14], [15], [16], [17], [18], [19] typically trigger alerts only upon the final, terminal request of an attack chain. This is because state-setting requests (e.g., modifying a template or updating a configuration) often appear benign and conform to legitimate API schemas. Detection occurs only when the final request triggers anomalous system behavior, such as unauthorized command execution or exfiltration via sensitive syscalls [20], [21].

However, the terminal request is merely the symptom, not the complete exploit. For security responders, the primary challenge is the reconstruction of the prerequisite attack sequence. Without the preceding state-setting requests, the terminal request is non-reproducible in a sandbox, rendering root cause analysis, patch verification, and threat hunting nearly impossible. In production environments, these critical precursor requests are buried within massive, high-velocity traffic logs, where the signal-to-noise ratio makes manual correlation impractical.

The security industry currently lacks an automated mechanism to bridge this gap. This technical deficit presents a critical challenge: *given only the terminal anomalous request, how can we efficiently and automatically reconstruct the complete state-setting sequence required to reproduce the attack?*

Previous Solutions and their Limitations: Automated web application testing has been extensively studied to fulfill this task. One line of work involves coverage-guided grey-box fuzzers [22], [23], which excel at exploring code paths but are goal-blind with respect to specific vulnerabilities. To improve efficiency, directed fuzzers were proposed to focus fuzzing efforts on reaching predefined, potentially dangerous code locations in web applications [24], [25]. However, the “target” for these tools is a single code location, not the complex application state required by multi-step attacks. They are designed to craft an input within a single request to reach a code sink, but they lack the ability to reason about and construct the state-setting prefix sequence that spans multiple requests. Consequently, they remain inefficient for reconstructing stateful attack sequences.

A more relevant line of work focuses on fuzzing sequences of RESTful API calls [4], [26], [27], [28]. These tools aim to discover deep vulnerabilities by generating longer request sequences. Some approaches are data-driven, assuming longer

sequences are more likely to uncover bugs [27]. Others rely on manually annotated API specifications [4] or infer API dependencies through keyword matching and RESTful semantics [28]. Despite their advances, these methods have fundamental limitations in our problem context.

- *Lack of Goal-Orientation:* Existing sequence generation strategies, whether driven by sequence length [27], manual annotations [4], or keyword heuristics [28], are unable to leverage the invaluable information of the final, required attack state. Their exploration is akin to a blind search rather than a guided pathfinding process.
- *State-Blind Parameter Generation:* These methods typically populate parameters by learning from previous requests and responses, making them blind to the backend application state. They overlook a critical fact in our scenario: many necessary parameter values, such as an injected command or a specific ID, already exist in the database or file system as a result of the attack. The challenge is not to generate a new value from scratch, but to trace the correct, existing value from the backend state and associate it with the right request parameter.
- *Reliance on Rigid Semantics:* Some approaches heavily rely on expert domain knowledge for annotation [4], which is costly and hard to generalize. Others depend on idealized RESTful design conventions to infer API dependencies [28], a model that frequently fails in practice as many real-world applications do not adhere to these standards.

Our Solution: To address these challenges, we propose ARIADNE, a novel goal-oriented fuzzing framework designed to automatically reconstruct the prerequisite request sequences for stateful web attacks. The core idea of ARIADNE is to formalize the reconstruction problem as a guided search process, where the “goal” is the required backend state for the final attack and the feedback is a “state distance” metric. ARIADNE follows a grey-box fuzzing paradigm [22], [23]: it collects rich execution feedback (syscalls, database query logs) and network-level observations without requiring access to application source code.

Starting from the known final attack request, ARIADNE first performs a dynamic analysis to precisely characterize the required data dependencies on the backend, which we call the *target state*. It then iteratively mutates request sequences and uses our novel state distance metric to evaluate whether a new sequence brings the application closer to the target state, thereby efficiently guiding the entire exploration. To realize this vision, ARIADNE introduces a set of complementary techniques to overcome three core technical challenges.

1) *State Distance Navigation:* To provide precise guidance, ARIADNE first identifies the exact data dependencies of the final attack request by analyzing its syscalls, creating a *target state portrait*. We then introduce a novel state distance metric based on the minimum weight matching in a bipartite graph. This metric accurately quantifies the difference between the current state and the target state, providing a clear navigational signal for the fuzzer.

2) *Goal-Oriented API Selection:* To efficiently construct sequences, ARIADNE builds a data-driven API dependency graph that captures the actual probabilistic succession between APIs.

When selecting the next API, it employs a multi-heuristic planning process that not only finds the shortest path to the target state on this graph but also pre-checks the parameter satisfiability of the chosen path, avoiding futile exploration.

3) *State-Driven Parameter Sourcing:* To solve the parameter generation problem, ARIADNE incorporates the backend state (e.g., database, file system) as a source for parameter values. By analyzing the similarity between various data sources and API parameters, it can accurately trace and backfill information left in the system by the attack into the prerequisite requests, dramatically increasing the hit rate of generating valid and effective requests.

Experiments and Evaluations: We implemented a prototype of ARIADNE and conducted a comprehensive evaluation on a benchmark of 15 real-world, stateful vulnerabilities (CVEs/CNVDs) from 13 widely-used open-source applications. The experimental results demonstrate that ARIADNE consistently reconstructed all target attack sequences within the benchmark. All reconstructions were completed in under 20 minutes, including a complex attack chain requiring 8 steps. Furthermore, as a secondary finding, ARIADNE discovered a shorter and more efficient attack path than the one described in the public proof-of-concept in 5 cases. In contrast, state-of-the-art baseline tools, even when given a significant advantage by being provided with the necessary attack payloads, failed to reconstruct any attack requiring three or more steps.

Our Contributions:

- We are the first to systematically formalize the critical security task of reconstructing prerequisite attack sequences as an automated, goal-oriented state search problem, and we propose ARIADNE, a complete framework to solve it.
- We design a precise guidance system centered on state distance. It constructs a target state portrait by analyzing syscalls and employs minimum weight bipartite matching to quantify state distance, providing reliable navigation for efficient exploration in complex web environments.
- We implement a state-driven sequence generation engine. It combines a data-driven API dependency graph with a smart planning algorithm for efficient API selection and a novel parameter sourcing mechanism that traces values from the backend state, enabling the efficient reconstruction of the attacker’s original information flow.
- We conduct a comprehensive evaluation on a benchmark of 15 real-world, multi-step vulnerabilities. The results demonstrate that ARIADNE consistently reconstructs all target attack sequences. This performance decisively outperforms state-of-the-art baselines, which fail on all but the simplest scenarios.

In the spirit of open science, we make our tool available at <https://github.com/horis-dev/Ariadne>

II. RELATED WORK

Fuzzing for RESTful APIs: Prior work on fuzzing RESTful APIs has focused on generating valid request sequences to uncover deep vulnerabilities. MINER [27] is a data-driven fuzzer that generates long sequences, assuming length correlates with

bug discovery. To introduce more targeted testing, VoAPI [28] infers API dependencies using keyword matching and RESTful semantics, but its reliance on idealized conventions limits its applicability to real-world applications. NAUTILUS [4] achieves high-quality sequences by leveraging manually annotated API specifications, but this approach incurs significant human effort and is hard to generalize. These approaches are goal-blind; they either explore the state space without a specific target or rely on fragile heuristics. In contrast, ARIADNE leverages the final attack state to perform a guided search, transforming the problem into an efficient pathfinding task.

Grey-box and Directed Fuzzing for Web Applications: Grey-box and directed fuzzing for web applications aim to improve testing efficiency by using program feedback. Grey-box fuzzers like WITCHER [22] and ATROPOS [23] pioneered the use of code coverage to explore web application logic. However, their exploration is undirected with respect to specific vulnerabilities. To improve focus, directed fuzzers such as PREDATOR [25] for PHP and WDFUZZ [24] for Java guide test generation towards reaching predefined dangerous code locations. While effective for single-request exploits, the fundamental limitation of these tools is that their target is a code location, not a complex application state. They are designed to craft a single input to reach a sink and cannot construct the multi-request, state-setting sequences our work focuses on.

III. BACKGROUND

A. Bipartite Matching for State Distance

Many problems in computer security require quantifying the similarity or dissimilarity between two structured configurations. When each configuration can be represented as a *set of entities* (e.g., files, database rows, configuration entries), a natural way to define their distance is to ask: what is the cheapest way to transform one set into the other? This question can be formalized as a classical combinatorial optimization problem over *bipartite graphs*.

Given two disjoint vertex sets U and V , a *complete weighted bipartite graph* $B = (U \cup V, E, W)$ contains an edge $(u, v) \in E$ for every pair $u \in U, v \in V$. Each edge carries a non-negative weight $w(u, v)$ that represents the cost of associating entity u with entity v . A *perfect matching* is a subset of edges $M \subseteq E$ such that every vertex in $U \cup V$ is incident to exactly one edge in M (assuming $|U| = |V|$, which can always be ensured by adding dummy nodes). The *minimum-weight perfect matching* is the matching that minimizes $\sum_{(u,v) \in M} w(u, v)$. This problem is solvable in polynomial time, e.g., by the Hungarian algorithm or a min-cost max-flow formulation [29], [30].

In the context of stateful attack reconstruction, we treat the current application state and the target state (the state required by the final exploit) as two entity sets. Each entity corresponds to a database row or a file, as described in Section IV-B1. The edge weight $w(u, v)$ represents the effort needed to transform entity u into entity v —for example, updating a string field or replacing a file. The total cost of the minimum-weight matching therefore gives a rigorous, quantitative measure of the *state distance* between the two configurations. Section IV-C3 details how we instantiate this general framework for web applications.

B. Motivation

To illustrate the operational challenges of attack reconstruction, we consider a real-world Remote Code Execution (RCE) incident in CMS Made Simple (CVE-2021-26120). This exploit is a classic multi-step stateful attack. First, an adversary sends a POST request to inject a malicious Server-Side Template Injection (SSTI) payload into the application’s database. Second, a seemingly benign GET request to the homepage triggers the RCE by rendering the previously polluted template.

The Reconstruction Gap: In a production environment, an IDS typically flags the incident only upon the second request, detecting the anomalous shell spawn. However, the captured GET `/index.php` request contains no malicious payload. When a security responder attempts to replay this request in a forensic sandbox, the exploit fails to trigger because the environment lacks the prerequisite backend state (the SSTI payload). To perform root cause analysis or verify a patch, the responder must identify the specific state-setting POST request—a task that is practically impossible when the request is buried among millions of legitimate historical traffic logs.

Limitations of Existing Approaches: Existing automated tools fail at this task due to their goal-blind and state-blind nature. In this scenario, they cannot intelligently select the specific API for template editing from dozens of candidates. Nor can they generate the required complex SSTI payload, as it is neither trivial to create from scratch nor available in prior, unrelated responses.

Our Approach with ARIADNE: ARIADNE’s goal-oriented design transforms this blind exploration into a guided pathfinding problem.

1) *Constructing a Target State Portrait:* ARIADNE begins by dynamically analyzing the final attack request (GET `/index.php`). By tracing its syscalls, it precisely identifies its backend data dependency: a read from the database via a query like `SELECT * FROM cms_layout_templates WHERE id=10`. Crucially, ARIADNE extracts the specific content read from this row—the malicious SSTI payload. This “polluted database row” is captured to form the target state portrait, defining the goal of the reconstruction.

2) *Goal-Oriented Sequence Generation:* This goal directs the entire reconstruction process. The API selection strategy prioritizes APIs that can write to the `cms_layout_templates` table (the API for editing templates), and the parameter sourcing mechanism then traces the required SSTI payload directly from the target state portrait to populate the parameters of the prerequisite request.

This goal-oriented approach is highly effective. In our experiments, ARIADNE not only reconstructs the attack for this CVE in under three minutes on average but also discovers a more optimal, shorter attack path than the one described in the public proof-of-concept [21], demonstrating its ability to efficiently navigate the state space.

C. Threat Model

We target multi-step, state-dependent web attacks where an adversary manipulates an application’s persistent backend state (e.g., database, file system) to enable a final exploit. ARIADNE

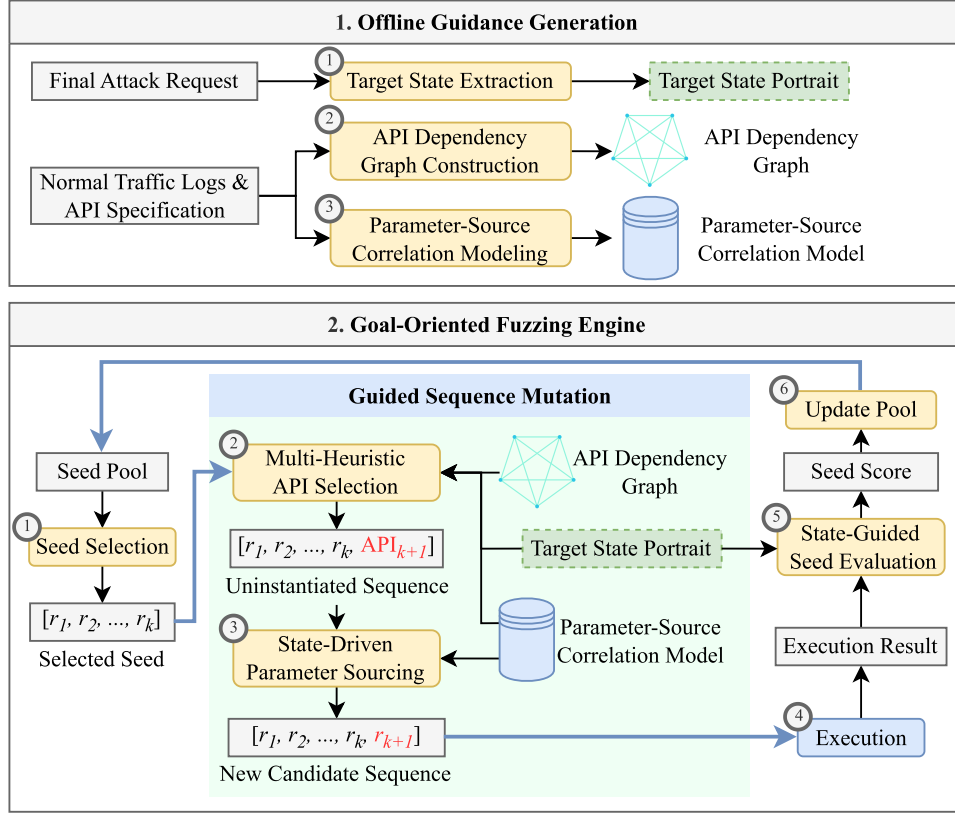


Fig. 1. The overall architecture of ARIADNE.

is designed for a post-incident forensic scenario: an attack has already occurred, and while the enterprise’s detection infrastructure has flagged the final malicious request, the preceding state-setting requests remain unidentified within the system logs.

Assumptions: Our analysis is based on a post-mortem audit workflow and relies on the following assumptions:

- **Input and Availability:** We assume the analyst has identified the terminal, payload-triggering http request via an IDS alert or SIEM log. In addition, the defender is assumed to have access to (i) low-level execution traces (syscalls) of the terminal request [6], [12]; (ii) historical normal traffic logs, e.g., from API gateways or staging environments; and (iii) backend storage snapshots or audit logs (such as database query logs) for the purpose of building guidance models. These assumptions are consistent with a *grey-box* testing methodology [4], [22], [23], [25], where the application is instrumented to provide rich feedback while source code access is not required.
- **Detection Lag:** We assume a common industrial reality: the prerequisite state-setting requests do not trigger immediate alerts. Because these requests often involve legitimate API operations, they are only identified as malicious in hindsight, once their cumulative effect is realized by the terminal request.
- **State Persistence:** We focus on attacks that achieve their goals by modifying persistent backend storage. The success of the terminal request must be contingent on reading

or executing artifacts (e.g., polluted database rows, uploaded scripts) created by the preceding steps.

- **System Trustworthiness:** We assume the underlying operating system and its auditing framework are part of the Trusted Computing Base (TCB). The audit logs, normal traffic logs used for behavioral modeling, and the ARIADNE framework itself are protected from tampering, consistent with established log-based forensic models [31], [32].

Scope: ARIADNE focuses on reconstructing attack sequences initiated via external http requests that tamper with persistent backend state. Out of scope are attacks with purely transient state (e.g., in-memory), non-application layer exploits, attacks not aimed at state modification (e.g., DoS, side-channel attacks), and insider threats. This scope aligns with related work on stateful web security analysis [4], [27], [28].

IV. SYSTEM DESIGN

A. System Design Overview

The core objective of ARIADNE is to automatically reconstruct the prerequisite state-setting sequence for a stateful attack, given only its final request. To achieve this, we design a goal-oriented fuzzing framework. As illustrated in Fig. 1, its architecture consists of two main phases: an offline guidance generation phase and an online, goal-oriented fuzzing engine.

Offline Guidance Generation: In the offline phase, ARIADNE performs a one-time analysis of the final attack’s execution

trace and normal application traffic to build three key guidance models. These models steer the online fuzzer.

- 1) *Target State Portrait*: A *Target State Portrait* is derived by dynamically analyzing the final request to extract its precise data dependencies on the backend database and file system. This portrait defines the fuzzer's ultimate goal.
- 2) *API Dependency Graph*: A data-driven *API Dependency Graph* models the succession probabilities between APIs. It is built by analyzing value similarity in request-response pairs and shared backend state access patterns from normal traffic, overcoming the reliance on rigid semantics.
- 3) *Parameter-Source Correlation Model*: A *Parameter-Source Correlation Model* addresses the parameter-filling challenge by identifying the likely origin of a parameter's value. It computes the Jaccard similarity [43] between the value sets of parameters and various data sources.

Goal-Oriented Fuzzing Engine: The online fuzzer is an iterative loop driven by the guidance models. It repeatedly mutates a request sequence (a seed) via a *Guided Sequence Mutation* process, which involves two key steps:

- 1) *Multi-Heuristic API Selection*: This strategy intelligently chooses the next API to append by weighing three factors: the transition probability on the API dependency graph, the path potential towards the target state, and the immediate parameter satisfiability.
- 2) *State-Driven Parameter Sourcing*: This mechanism populates the selected API's parameters by tracing and sourcing values from prior responses, the target state portrait, and an initial corpus, guided by the correlation model.

The mutated sequence is executed in an isolated environment. Its quality is then evaluated by a *State-Guided Seed Evaluation* module. The core of this evaluation is a novel *State Distance* metric, calculated as the minimum weight matching cost in a bipartite graph between the current and target states. This distance, combined with traditional metrics like code coverage, produces a composite score used by a simulated annealing scheduler to update the seed pool.

This closed loop of mutation, execution, and evaluation allows ARIADNE to systematically narrow the distance to the target state until the full attack chain is reconstructed.

B. Offline Guidance Generation

In the offline phase, ARIADNE pre-computes three guidance models from the final attack's trace and normal traffic logs to steer the fuzzer.

1) *Target State Extraction*: The objective of this module is to construct a precise *Target State Portrait* for the final attack request. This portrait details all the specific backend data that the request relies upon for successful execution. Its inputs are the final attack request and the system-level audit logs, such as syscall traces, collected during its execution. To accurately map a high-level http request to its low-level data dependencies, we adapt established attack provenance analysis techniques [6], [44], [45]. The process involves two main steps.

1) *Isolating System Activity*: We first inject lightweight probes into the web application framework to log markers with a

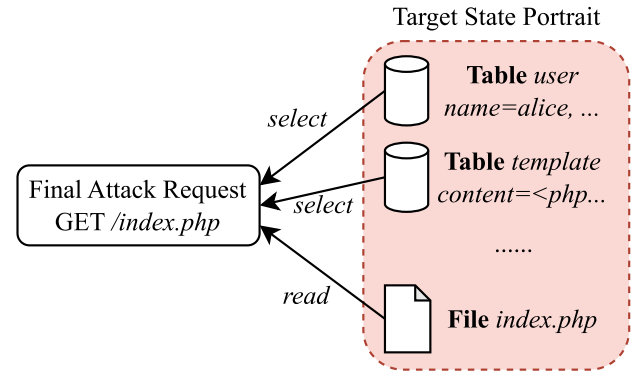


Fig. 2. An example of a Target State Portrait.

unique request ID at key points in each request's lifecycle. These markers allow us to deterministically partition the global syscall log and isolate the complete syscall sequence belonging exclusively to the final attack request. This sequence provides a comprehensive record of all underlying activities triggered by the request.

2) *Extracting Data Dependencies*: With the corresponding syscall sequence isolated, we then extract its read dependencies on files and the database.

- *File Dependencies*: We analyze file-related syscalls (e.g., `openat`, `read`) to identify the paths of files read by the request and the specific content retrieved.
- *Database Dependencies*: Extracting database dependencies is more complex as it spans the application and database servers. We first identify the network connection (via its network tuple) and its active time window from the request's syscall sequence. We then correlate this information with the database server's logs to pinpoint the specific worker process. Finally, by inspecting this worker's writes to the database's query log file, we extract the executed `SELECT` queries and their result sets.

The final output is a structured target state portrait (Fig. 2). To facilitate a unified state metric, ARIADNE employs a *pluggable state normalization layer* that abstracts all dependencies into a common, table-based representation. Specifically, database entities retain their (table, row) structure, while file dependencies are mapped to a conceptual files table with a (filename, content) schema (e.g., row: {"filename": "/etc/config.xml", "content": "..."}).

This homogenization is achieved through lightweight, application-specific adapters that parse raw storage artifacts (e.g., SQL result sets, JSON configs) into this unified schema. Such a decoupled design ensures that the high-level state reasoning and distance calculation remain invariant across different application stacks, while only the adapters need to be tailored to the structural diversity of the backend storage. This unified model is crucial for the subsequent search process, as it enables a generic bipartite matching approach to provide a clear, quantifiable navigational signal.

2) *Data-Driven Guidance Model Construction*: To steer the fuzzer, ARIADNE constructs two data-driven models from normal

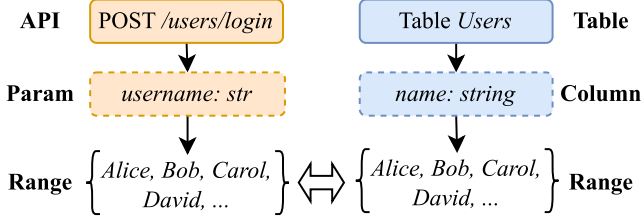


Fig. 3. An example of Parameter-Source Correlation.

traffic logs and backend snapshots. These models capture the empirical data-flow and API succession patterns, bypassing the limitations of rigid, idealized RESTful semantics.

Parameter-Source Correlation Model: Attack reconstruction requires populating parameters with context-aware values (e.g., session tokens, entity IDs). We formalize this as a mapping between a request parameter p and a *Data Source Position* s (e.g., a response field or a database column).

We collect value sets $V(p)$ and $V(s)$ from historical traffic and database snapshots, then quantify their relationship using Jaccard similarity [43]: $Score_{corr}(p, s) = |V(p) \cap V(s)| / |V(p) \cup V(s)|$. A high score indicates a likely data-flow dependency (Fig. 3). This model enables the fuzzer to backfill parameters by sourcing values from the backend state or prior responses rather than generating them from scratch.

API Dependency Graph (ADG): The ADG is a directed weighted graph $G = (V, E, W)$ where nodes are API endpoints and edge weights $W(u, v)$ represent succession likelihood. ARIADNE infers these weights by fusing two signals:

- **Direct Flow (P_{flow}):** Represents explicit chaining. For APIs u and v , $P_{flow}(u, v)$ is the normalized maximum correlation score between any response field of u and any request parameter of v : $\max Score_{corr}(v.req_j, u.resp_i)$.
- **Indirect State Flow (P_{state}):** Captures asynchronous dependencies mediated by storage. Using the syscall-level analysis described in Section IV-B1, we identify the set of entities written by u ($Writes(u)$) and read by v ($Reads(v)$). The dependency strength $P_{state}(u, v)$ is proportional to $|Writes(u) \cap Reads(v)|$.

The final transition probability is a weighted fusion:

$$W(u, v) = \alpha P_{flow} + (1 - \alpha) P_{state} \quad (1)$$

as illustrated in Fig. 4. This ensures the ADG captures both observable API chaining and hidden, state-mediated interactions. Note that both P_{flow} and P_{state} capture semantic correspondence (value overlap) and shared storage dependencies rather than raw invocation frequencies. Hence, an API that is rarely called but semantically relevant still receives an appropriate weight, while a frequently invoked but unrelated API does not bias the graph.

C. Goal-Oriented Fuzzing Engine

With the offline guidance models constructed, ARIADNE's goal-oriented fuzzing engine begins its operation. The engine frames the reconstruction task as an iterative search problem,

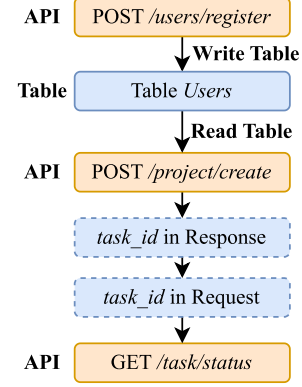


Fig. 4. An example of API Dependency.

aiming to find a sequence of requests that leads to the *Target State Portrait*. It operates via a continuous fuzzing loop, which orchestrates the core modules for guided sequence mutation and state-guided seed evaluation to systematically explore the application's state space.

1) **Fuzzing Loop and Scheduling:** The fuzzing loop iteratively selects a seed sequence using a power scheduler [46], [47]. The seed is mutated via *Guided Sequence Mutation* (Section IV-C2) and the resulting candidate is executed. Its outcome is evaluated by our *State-Guided Evaluation* module (Section IV-C3). A simulated annealing policy [46] then decides whether to add the candidate to the seed pool, balancing exploitation and exploration.

2) **Guided Sequence Mutation:** To reconstruct the attack chain, ARIADNE extends a seed sequence S by appending a new, carefully selected request. This process involves: (1) selecting the most promising next API to call, and (2) populating its parameters with semantically valid values.

Multi-Heuristic API Selection: To prune the combinatorial search space, ARIADNE employs a multi-heuristic strategy to rank candidate APIs. For a current sequence $S_k = (a_1, \dots, a_k)$ and a candidate API b , the composite score is the weighted product of three complementary heuristics:

$$Score(b|S_k) = [H_{trans}]^{r_1} \cdot [H_{goal}]^{r_2} \cdot [H_{app}]^{r_3} \quad (2)$$

This scoring mechanism balances historical patterns, goal-orientation, and immediate feasibility. The three heuristics are detailed below:

1) **Transition Heuristic (H_{trans}):** This heuristic evaluates the statistical likelihood of b succeeding S_k based on the ADG. We aggregate transition probabilities $W(a_i, b)$ using an exponential decay function $\sum_{i=1}^k W(a_i, b) \cdot e^{-\lambda(k-i)}$, prioritizing APIs that align with recent operations and common call patterns.

2) **Goal-Oriented Heuristic (H_{goal}):** This heuristic forcefully steers exploration toward the target state. We first identify *Target APIs* (A_{target}) capable of writing the missing entities in the current state. H_{goal} then calculates the probability of the most likely path from b to any $t \in A_{target}$ on the ADG.

To compute H_{goal} efficiently, we formalize the max-product path problem on the ADG. Given a candidate API b and a set of

Algorithm 1: Goal-Oriented Heuristic Score Computation.

Input : API graph G , Target API set T
Output: Goal heuristic scores for all APIs

```

1  $D \leftarrow$  a map from API to distance, init  $\infty$ ;
2  $Q \leftarrow$  a priority queue;
3 foreach  $t \in T$  do
4    $D[t] \leftarrow 0, Q.\text{INSERT}(t, 0)$ ;
5 end
6 while  $Q$  is not empty do
7    $u \leftarrow Q.\text{EXTRACTMIN}()$ ;
8   foreach predecessor  $v$  of  $u$  in  $G$  do
9      $w \leftarrow -\log W(v, u)$ ;
10    if  $D[v] > D[u] + w$  then
11       $D[v] \leftarrow D[u] + w$ ;
12       $Q.\text{UPDATEPRIORITY}(v, D[v])$ ;
13    end
14  end
15 end
16 return  $\{api : e^{-D[api]} \text{ for each api}\}$ 

```

target APIs A_{target} , we seek:

$$\max_{t \in A_{target}} \left(\max_{\text{path}(b \rightsquigarrow t)} \prod_{e_i \in \text{path}} W(e_i) \right) \quad (3)$$

We transform this into a shortest-path problem by defining new edge weights $W'(e) = -\log(W(e))$. The product of probabilities thus becomes a sum of log-weights. We run a multi-source Dijkstra algorithm on the reversed ADG, starting from all $t \in A_{target}$ with initial distance 0. The resulting shortest distance $D[b]$ gives the goal heuristic score $H_{goal} = e^{-D[b]}$. The complexity $O(E \log V)$ is negligible for typical API ecosystems, enabling real-time guidance during fuzzing. This ensures the fuzzer prioritizes “stepping-stone” APIs that bridge the gap to the final exploit. Algorithm 1 details this process.

3) *Applicability Heuristic (H_{app})*: This heuristic acts as a feasibility filter. It estimates whether the parameters of candidate b can be satisfied using values available in current data sources. H_{app} is calculated as the minimum satisfiability score across all parameters of b based on semantic correlation and source priority.

To ensure a candidate API b is feasible, H_{app} assesses the likelihood of its parameters being populated with high-confidence values. For a parameter $p \in b.params$, its satisfiability is defined as the highest score among all available values in the current environment:

$$\text{Satisfiability}(p|S_k) = \max_{s \in \text{Corpus}(S_k)} (\text{Score}_{corr}(p, s) \cdot \omega(s)) \quad (4)$$

where $\text{Score}_{corr}(p, s)$ is the Jaccard similarity from the correlation model, and $\omega(s)$ is the weight of the source corpus. The heuristic for the API is then:

$$H_{app} = \min_{p \in b.params} \text{Satisfiability}(p|S_k) \quad (5)$$

This prevents the fuzzer from selecting APIs that are strategically sound but practically uncallable due to missing dependencies.

By fusing these heuristics, ARIADNE selects an API that is statistically probable, strategically relevant, and immediately actionable.

State-Driven Parameter Sourcing: After selecting an API, ARIADNE must populate its parameters with semantically valid values. We reframe this as a *guided sourcing* problem rather than random generation. ARIADNE employs a *Three-Tier Corpus System* with varying priorities: (1) *Local Corpus*, containing values from prior responses in the current sequence; (2) *Target Corpus*, containing critical values extracted from the Target State Portrait (e.g., malicious SSTI payloads); and (3) *Initial Corpus*, containing general values from normal traffic.

To fill a parameter p , ARIADNE calculates a selection score for candidate values based on the Score_{corr} from our correlation model and the corpus priority. Once an API is selected, each parameter p is populated by sampling from the combined corpora. For a candidate value v from source s , its selection probability $P(v|p)$ is normalized using a roulette-wheel strategy:

$$P(v|p) = \frac{\text{Score}_{corr}(p, s) \cdot \omega(s)}{\sum_{(v', s') \in \text{AllCorpora}} \text{Score}_{corr}(p', s') \cdot \omega(s')} \quad (6)$$

The weights follow $\omega_{local} > \omega_{target} > \omega_{initial}$, ensuring that context-aware and goal-oriented values are prioritized while maintaining a non-zero probability for exploratory fallback values.

This mechanism allows ARIADNE to precisely backfill the attacker’s original information flow by tracing values directly from the required backend state.

3) *State-Guided Seed Evaluation*: To quantitatively evaluate each new candidate sequence, ARIADNE employs a state-guided evaluation mechanism. It uses a comprehensive scoring function that balances our novel state distance metric with traditional exploration metrics. This provides a feedback signal that balances goal-orientation with the need to escape local optima.

State Distance Metric: The state distance metric quantifies the gap between the current state $V(S_k)$ and the target state portrait G . We model this distance as the minimum transformation cost required to convert the current state into the target state.

To this end, we construct a weighted complete bipartite graph $B = (U \cup V, E, W)$. The node set U on one side represents all data entities (i.e., database rows or files) in the current state $V(S_k)$, while the node set V on the other side represents entities in the target state G . As described in Section IV-B1, state entities are normalized into a uniform, table-based representation. For each pair of nodes $u \in U$ and $v \in V$, an edge exists with a weight $w(u, v)$ representing the cost of transforming entity u into entity v . The cost is calculated based on whether u and v are of the same type.

- *Same-Type Entities*: If u and v belong to the same table, their transformation cost is the sum of the distances between their corresponding attributes. We define attribute-specific distance functions d_{attr} : for numeric types, it is the absolute difference; for strings, it is the normalized Levenshtein distance; for boolean or enum types, it is 0 if the values are identical and 1 otherwise. The cost is

formally defined as:

$$w(u, v) = \sum_{i \in \text{columns}} d_{attr}(u.col_i, v.col_i) \quad (7)$$

- *Different-Type Entities*: If u and v do not belong to the same table, we consider a direct transformation impossible. The operation is modeled as a deletion of u and a creation of v , incurring a large, fixed penalty weight $C_{mismatch}$ (set to 2 by default; a sensitivity analysis is provided in Section VI-E).

After constructing this graph, we compute the total weight of its minimum weight perfect matching. This total weight, denoted as $\text{Dist}(V(S_k), G)$, represents the minimum cost to transform the current state entities into the target state entities. This problem can be solved efficiently in polynomial time using standard algorithms like the Hungarian algorithm [29] or a min-cost max-flow formulation [30].

Comprehensive Seed Scoring: While the state distance metric is the primary guide, relying on it exclusively can trap the search in local optima. To balance goal-orientation with effective exploration, ARIADNE employs a comprehensive scoring function that integrates our state-aware metric with traditional fuzzing incentives. The composite score for a candidate sequence S' is a weighted sum of three components:

$$\text{Score}(S') = w_s R_{state}(S') + w_c R_{cov}(S') + w_v R_{val}(S') \quad (8)$$

where w_s, w_c, w_v are hyperparameters balancing the importance of each term.

- 1) *State Distance Reward* (R_{state}): This is the core component, rewarding sequences that reduce the distance to the target state. It is defined as an exponential function of the state distance computed previously:

$$R_{state}(S') = e^{-k_s \cdot \text{Dist}(V(S'), G)} \quad (9)$$

where k_s is a scaling factor.

- 2) *Coverage Gain Reward* (R_{cov}): This is a fuzzing incentive that rewards sequences for discovering new code paths (e.g., basic blocks, edges), encouraging broader exploration of the application's logic.
- 3) *Response Value Range Reward* (R_{val}): This reward encourages the discovery of new internal application states by rewarding sequences that produce previously unseen values in their responses, such as new error codes or unique identifiers.

V. IMPLEMENTATION

We implemented a prototype of ARIADNE in over 9,000 lines of Python code. Our implementation is a self-contained framework that addresses three key practical challenges: ensuring scalable execution, handling dynamic session data, and normalizing heterogeneous backend states.

Scalable and Isolated Execution Environment: To ensure state isolation and enable high-throughput testing, ARIADNE leverages a pool of pre-warmed Docker containers. Each candidate sequence is executed in a dedicated, clean container instance, which is destroyed after run. This model guarantees each test

starts from a pristine state, preventing interference between cases.

Handling Dynamic Application States: To overcome the challenge of ephemeral values like session IDs or CSRF tokens, ARIADNE employs a *Symbolic Reference Mechanism*. Instead of storing concrete values, it saves placeholders within the sequence (e.g., `<req1.response.body.csrf_token>`). A runtime agent resolves these references by extracting the correct, fresh values from the current execution context, ensuring sequence re-playability.

State Abstraction and Adapters: To handle diverse backend storage formats, ARIADNE automatically extracts state entities from standard relational databases and file systems using built-in generic parsers. For applications using semi-structured data (e.g., nested JSON) or NoSQL stores, ARIADNE exposes a standardized interface that requires minimal application-specific logic (averaging 58 LoC) to map raw storage into our unified table-based representation. This design ensures that ARIADNE's core state-search engine remains agnostic to the underlying storage technology. As discussed in Section VII, building these adapters is a one-time setup cost per application.

Automated Attack Verification Oracle: To automatically confirm a successful reconstruction, our oracle executes the final attack request A_{final} in the state produced by a candidate sequence. It then verifies success by checking for pre-defined indicators, such as specific strings in the final response (e.g., `uid=0(root)`) or expected malicious changes in the backend state (e.g., upload of a webshell). This allows ARIADNE to reliably detect when the goal is reached.

VI. EVALUATION

We designed our experiments to answer the following research questions:

- *RQ1*: How effective and efficient is ARIADNE in reconstructing complex, state-dependent web attack sequences compared to state-of-the-art baselines?
- *RQ2*: What is the contribution of ARIADNE's core design components—the state distance metric, multi-heuristic API selection, and state-driven parameter sourcing—to its overall performance?
- *RQ3*: How do ARIADNE's internal guidance mechanisms work together to guide the fuzzing process in a real-world, long-chain attack scenario?
- *RQ4*: How sensitive is ARIADNE's performance to its key hyperparameters?

A. Evaluation Setup

Benchmarks: To evaluate ARIADNE in realistic scenarios, we constructed a benchmark suite of 15 vulnerabilities (Table I). The selection process was governed by a systematic filtering of the Vulhub repository [48] as well as prior work on web application fuzzing (e.g., MINER [27] and VOAPI [28], detailed in the next paragraph). In addition, we incorporated two recent high-severity CVEs (Kanboard, CVE-2025-46825 [49], and Admidio, CVE-2025-62617 [50]) directly obtained from

TABLE I
BENCHMARK SUITE DETAILS AND COMPLEXITY METRICS

Application	Vulnerability ID	API Count	Parameter Count	Corpus Type Count	Reported Steps	Vulnerability Type
Apache Solr	CVE-2019-17558	21	46	3202	2	RCE
GitLab	CVE-2016-9086	26	49	284	5	Arbitrary File Read
pgAdmin	CVE-2023-5002	22	44	114	5	RCE
ActiveMQ	CVE-2016-3088	23	19	43	3	Arbitrary File Upload
Airflow	CVE-2020-11978	26	48	101	3	RCE
CMSMS	CVE-2021-26120	21	39	109	4	RCE
Elasticsearch	CVE-2014-3120	20	36	1096	2	RCE
CouchDB	CVE-2017-12636	19	21	345	4	RCE
DedeCMS	CNVD-2018-01221	24	53	132	3	Arbitrary File Upload
YApi	YApi Unacc [51]	18	44	276	8	RCE
Kanboard	CVE-2025-46825	28	26	85	5	XSS
Admidio	CVE-2025-62617	25	29	98	3	SQLi
GitLab (from MINER)	-	32	49	632	5	Improper State Manage
Appwrite (from VoAPI)	CVE-2022-2925	26	44	339	5	XSS
GitLab (from VoAPI)	CVE-2022-2230	38	64	612	6	XSS

their official GitHub repositories; these are absent from Vulhub and prior studies.

1) *Selection from Public CVEs*: We systematically reviewed multi-step attack scenarios in Vulhub. To ensure the benchmarks represent non-trivial, state-dependent challenges, we applied the following exclusion criteria: (i) we excluded trivial sequences that only require a single login or a CSRF token fetch; (ii) we excluded vulnerabilities using non-HTTP protocols (e.g., WebSockets); and (iii) we excluded exploits that are inherently unstable (e.g., those relying on precise timestamp-based shell access). This process yielded 10 diverse, high-impact vulnerabilities. Second, to capture the latest threat landscape, we further included two 2025 CVEs sourced directly from GitHub, namely Kanboard (CVE-2025-46825) [49] and Admidio (CVE-2025-62617) [50], which are multi-step attacks absent from Vulhub and prior work. We observed that all selected cases rely on persistent backend state transitions, fitting our research scope without requiring out-of-scope techniques like race conditions.

2) *Selection from Prior Work*: To further assess generalizability and facilitate direct comparison, we incorporated 3 additional scenarios from MINER [27] and VoAPI [28]. Specifically, we selected all explicitly documented multi-step attacks from these papers that involve a chain length of ≥ 5 steps, representing the most complex scenarios handled by state-of-the-art tools.

3) *Normal Traffic Acquisition*: For each benchmark application, we generated a baseline of normal traffic by executing manual scripts that exercise the core business logic of the involved APIs (the total API count per application is listed in Table I). These scripts simulate typical user behaviors, such as registration, profile updates, and resource management. In a production environment, this data can be seamlessly substituted with existing QA test suites, staging logs, or grayscale traffic.

The complexity of our benchmarks is summarized in Table I. *Parameter Count* denotes the total number of input parameters across all APIs required to reconstruct the attack, representing the direct search space for the fuzzer. *Corpus Type Count* quantifies the diversity of the corpus derived from normal traffic; it counts the distinct “value sources” including request parameters

of all observed APIs and database columns, which ARIADNE leverages for state-driven parameter sourcing.

For the in-depth analyses in RQ2 and RQ4, we selected a representative subset of three vulnerabilities: CVE-2016-9086, CVE-2023-5002, and CVE-2016-3088. These cases were chosen to cover three distinct categories of vulnerabilities, specifically Arbitrary File Read, RCE, and Arbitrary File Upload. This ensures that our internal mechanism evaluations are validated across diverse attack types.

Baselines: We compare ARIADNE against two state-of-the-art web application fuzzers: PREDATOR [25], a directed fuzzer for PHP applications, and MINER [27], a sequence-generation fuzzer for RESTful APIs. We excluded other related works for practical reasons: NAUTILUS [4] requires extensive manual annotation; VoAPI [28] could not be successfully reproduced despite attempts and communication with the authors; ATROPOS [23] has a complex setup involving a VM kernel module; and the source code for WDFUZZ [24] was not available.

Comparison Methodology: Our problem setting, reconstructing a prerequisite sequence for a known attack, poses an asymmetric challenge for existing tools not designed for this task. To ensure a meaningful and fair comparison, we established the following protocol.

- 1) *Shared Task and Oracle*: All tools, including ARIADNE and the baselines, were given the same task: find the prerequisite sequence that enables the final attack. We used a unified, automated verification oracle (Section V) to judge success.
- 2) *Enhanced Setup for Baselines*: Running the baselines out-of-the-box failed to solve any of our complex benchmarks. To make the comparison possible, we created an “enhanced” setup for them. We manually analyzed the full exploit chain for each vulnerability and extracted all critical, complex parameter values (e.g., SSTI payloads, serialized malicious objects) required in the prerequisite steps. We then provided these values as a high-quality initial seed corpus to PREDATOR and MINER. This gave them a significant advantage, relieving them of the need to generate or discover the necessary attack payloads.

TABLE II
EFFECTIVENESS AND EFFICIENCY COMPARISON BETWEEN ARIADNE AND ENHANCED BASELINES. TTV IS TIME-TO-VULNERABILITY

Vulnerability ID	Path Length (Reported)	ARIADNE Path Length (Avg/Min)	TTV (s)	PREDATOR (Enhanced) Success	MINER (Enhanced) Success / TTV (s)
CVE-2019-17558	2	3.8 / 2	159.5	N/A	✓ / 5.3
CVE-2016-9086	5	4.6 / 4	334.6	N/A	✗
CVE-2023-5002	5	8.2 / 6	80.2	N/A	✗
CVE-2016-3088	3	3.8 / 3	50.4	N/A	✗
CVE-2020-11978	3	3.2 / 3	245.6	N/A	✗
CVE-2021-26120	4	3.6 / 3	182.2	✗	✗
CVE-2014-3120	2	6.6 / 4	142.0	N/A	✓ / 6.8
CVE-2017-12636	4	5.2 / 4	58.6	N/A	✗
CNVD-2018-01221	3	3.4 / 3	352.6	✗	✗
YApi Unacc	8	6.0 / 5	234.4	N/A	✗
CVE-2025-46825	5	5.6 / 5	480.2	N/A	✗
CVE-2025-62617	3	3.2 / 3	165.0	N/A	✗

Note: ✓ = Success, ✗ = Failure or Timeout (6h). N/A denotes Not Applicable, as PREDATOR only supports PHP applications.

TABLE III
SHORTER ATTACK PATHS DISCOVERED BY ARIADNE COMPARED TO DOCUMENTED WORKFLOWS

Case	API Path	Optimization
CVE-2016-9086 (GitLab)	GET /users/sign_in POST /users POST /import/gitlab_project GET /:u_name/:p_name/import/new	Bypassed the project initialization, as its generated namespace_id was non-essential for the import.
CVE-2021-26120 (CMSMS)	POST /admin/login.php POST /admin/moduleinterface.php GET /index.php	Avoided the clear lock step in the official tutorial.
YApi Unauthenticated RCE [51]	POST /api/user/reg POST /api/project/add POST /api/interface/add POST /api/plugin/advmock/save GET /mock/:project_id/:basepath/:path	Skipped the login request as registration auto-logs in the user; omitted several GET requests for retrieving fixed information.
GitLab (from MINER)	POST /api/v4/projects POST /api/v4/projects/:id1/fork POST /api/v4/projects POST /api/v4/projects/:id1/fork/:id2	Triggered the error by skipping the prerequisite fork of the second project.
CVE-2022-2230 (GitLab, from VoAPI)	POST /api/v4/projects POST /api/v4/projects/:id/deploy_keys	Replaced the 6-step UI workflow with a direct 2-step RESTful API attack.

3) ARIADNE's *Setup*: In all experiments, ARIADNE did not receive this advantage. It started with only the information learned from normal traffic and had to automatically discover and backfill these critical payloads from the target state portrait via its state-driven parameter sourcing mechanism.

All experiments ran in isolated, reset-on-use Docker container groups on a server with an AMD Ryzen 9 7950X 16-Core Processor and 32 GB Memory.

B. RQ1: Overall Effectiveness and Efficiency

Our evaluation assesses ARIADNE's effectiveness and efficiency based on success rate, path length, and time-to-vulnerability (TTV), followed by a direct comparison with baseline tools.

Experimental Protocol: To account for the non-deterministic nature of fuzzing, we repeated each experiment 10 times. For all

TABLE IV
PERFORMANCE COMPARISON ON BENCHMARKS FROM PRIOR WORK

Source	App.	Original Tool TTV	Length	ARIADNE TTV	Length
MINER	GitLab	143 min	5	1022 s	4.8 / 4
VoAPI	Appwrite	63 s	5	227.1 s	9 / 7
VoAPI	GitLab	109 s	6	742.2 s	2.4 / 2

quantitative results, we report the arithmetic mean across these 10 trials.

Effectiveness and Efficiency of ARIADNE: On our benchmark suite, ARIADNE achieved a 100% success rate, reconstructing all 15 stateful attack sequences (Tables II and IV). All reconstructions were completed in under 20 minutes. As a secondary finding, in 5 of the 15 cases, ARIADNE discovered shorter attack paths than those publicly documented. For example, it condensed a 6-step UI-based workflow for a GitLab vulnerability into a

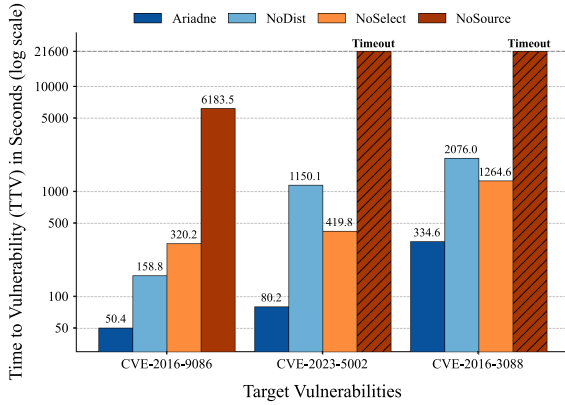


Fig. 5. Ablation study results showing the Time-to-Vulnerability (TTV) for different versions of ARIADNE.

direct 2-step API attack (Table IV). Details of all optimized paths are summarized in Table III.

Comparison with Baselines: We compared ARIADNE against PREDATOR and MINER, which were enhanced by providing them with all necessary complex attack payloads. As shown in Table II, these goal-blind tools struggled with sequence complexity. MINER solved only the two simplest 2-step attacks, and PREDATOR failed on both applicable multi-step scenarios. Both baselines were unable to reconstruct any attack requiring three or more steps, failing to manage the inter-request data dependencies.

For benchmarks from prior work (Table IV), we compared against the TTVs reported in their original papers. ARIADNE successfully reproduced all three vulnerabilities. While its TTV was higher for the two VOAPI benchmarks, this is attributable to our methodology. ARIADNE enforces strict, per-execution environment resets to ensure verifiable reconstruction, a process that is time-consuming for heavyweight applications like GitLab (85 s per reset). This trade-off, however, enabled the discovery of the more optimal 2-step attack path.

C. RQ2: Ablation Study

To quantify the contribution of each core component of ARIADNE, we conducted an ablation study. We created three degraded variants, each disabling one key mechanism by reverting it to a simpler, more conventional baseline strategy:

- **ARIADNE-NoDist:** Replaces our state distance metric for seed evaluation with traditional fuzzing feedback (i.e., code coverage and new value range discovery).
- **ARIADNE-NoSelect:** Replaces the multi-heuristic API selection with a uniform random selection from all available APIs.
- **ARIADNE-NoSource:** Replaces the prioritized, state-driven parameter sourcing with non-prioritized random sampling from the entire initial corpus.

We evaluated these variants against the full ARIADNE using the representative subset defined in Section VI-A under a 6-hour timeout. As shown in Fig. 5, the complete version of ARIADNE consistently and significantly outperforms all variants.

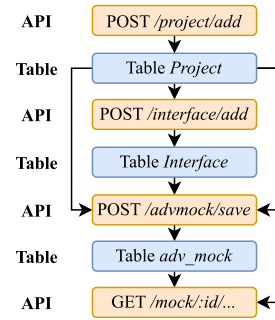


Fig. 6. The crucial final 4 steps of the YApi RCE discovered by ARIADNE, highlighting the data flow dependencies between API calls.

- **Without state distance feedback,** the fuzzer becomes largely goal-blind. The TTV increased dramatically across all benchmarks, with performance degrading by 3.2x (on CVE-2016-9086) to as much as 14.3x (on CVE-2023-5002).
- **Without heuristic API selection,** the search efficiency plummeted due to random exploration. This variant was 3.8x to 6.4x slower than the full ARIADNE, showing the importance of API selection.
- **The impact of removing state-driven parameter sourcing was the most severe:** Unable to construct semantically valid requests, this variant timed out on two of the three benchmarks and was over 123x slower on CVE-2016-9086 (6183.5 s vs. 50.4 s).

This study unequivocally demonstrates that ARIADNE's three core designs—state distance guidance, heuristic API selection, and state-driven parameter sourcing—are all indispensable and work synergistically to achieve efficient reconstruction.

D. RQ3: Case Study on a Long-Chain Attack

We use the complex YApi unauthenticated RCE vulnerability [51], which requires chaining user registration, project creation, and script injection, to illustrate ARIADNE's guided search.

Attack Reconstruction and Path Optimization: While the public PoC requires 8 steps [52], ARIADNE consistently found shorter paths (avg. 6, min. 5), reconstructing the attack in 234.4 s on average. This optimization stems from identifying and bypassing redundant requests, such as a superfluous login call, as it determined registration auto-logs the user in. Fig. 6 shows the final 4 critical steps and data dependencies.

Macro-level Fuzzing Dynamics: To provide a granular view, we analyze a representative run that found the 5-step path in 160 seconds (a 4-step prerequisite sequence). Fig. 7 plots the timeline, marking when ARIADNE successfully constructed a valid prefix of the final attack sequence. The process is not linear; instead, it consists of periods of exploration followed by rapid breakthroughs. For instance, ARIADNE spent the first 43 seconds exploring before constructing the initial POST /api/user/reg request. The subsequent discoveries of POST /api/project/add and POST

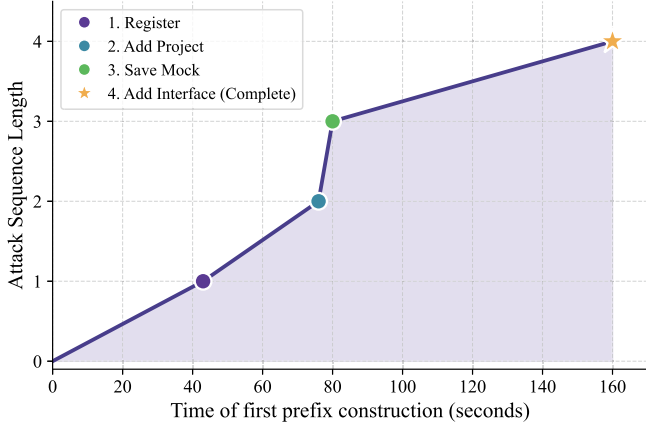


Fig. 7. Timeline of a 160-second run reconstructing the YApi attack. Each point marks the time a crucial state-setting request was successfully prepended to the sequence.

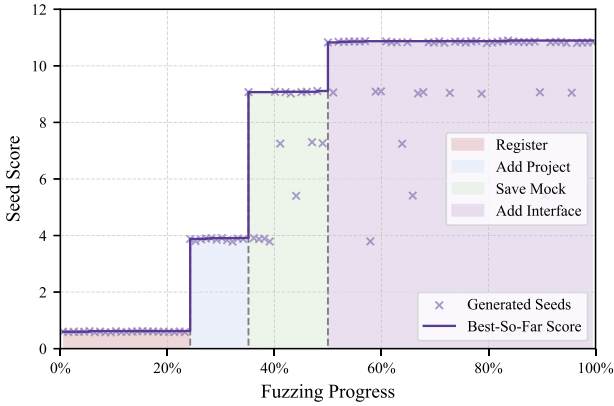


Fig. 8. Evolution of seed scores during the YApi reconstruction. Each point represents a newly generated candidate sequence, while the purple line shows the best-so-far score. The distinct phases demonstrate ARIADNE’s guided convergence toward the target state.

/api/plugin/advmock/save occurred in quicker succession at 76 s and 80 s, respectively, as the state space was constrained and guided by acquired state.

This guided convergence is driven by our state distance metric, quantitatively shown by the seed score evolution in Fig. 8. Each point represents a generated candidate sequence, with its score reflecting the proximity to the target state. The dark purple line traces the “best-so-far” score, which increases only when a superior sequence is discovered.

The plot reveals a “staircase” pattern, where each score increase corresponds to a breakthrough in the attack chain. For instance, creating the necessary user entity causes the best score to jump from a baseline of 0.8 to 3.9. Subsequent steps, like project creation and script injection, lead to further leaps to 9.1 and finally 10.9, signifying the discovery of the complete prerequisite sequence.

Dissecting a Guided Mutation Step: We dissect a pivotal mutation step to illustrate the synergy of ARIADNE’s guidance mechanisms. This step occurred after successfully creating a user via POST /api/user/reg, which had brought the best-so-far score to 3.9.

- *Heuristic API Selection:* The goal-oriented heuristic (H_{goal}) identified that creating a project was a critical prerequisite. This drove the selection mechanism to rank the correct API, POST /api/project/add, 3rd out of 18 candidates with 8.3% probability.
- *State-Driven Parameter Sourcing:* Once the API was selected, the sourcing mechanism was invoked to populate the `v_basepath` parameter. It correctly traced the value’s origin to the `basepath` field within the target state portrait’s `project` table, assigning this choice a top-ranked probability of 40.4%.

The successful execution of this guided request caused the best-so-far score to leap from 3.9 to 9.1, prompting the scheduler to prioritize this sequence and rapidly steer the search toward the complete solution.

E. RQ4: Hyperparameter Sensitivity

We analyze the sensitivity of ARIADNE to key hyperparameters using the same representative benchmarks defined in Section VI-A. Our analysis focuses on the API selection heuristic weights (r_1, r_2, r_3), the API dependency graph fusion factor (α), and the mismatch threshold parameter ($C_{mismatch}$).

Heuristic Weights: Fig. 9 shows the results of varying the heuristic weights. The key finding is ARIADNE’s robustness: it successfully found the vulnerability in all test cases. While the goal-oriented heuristic (H_{goal} , weighted by r_2) is the most influential—as increasing its weight generally lowers TTV—an over-reliance on any single heuristic degrades performance. This highlights the benefit of our balanced approach. Crucially, the TTV fluctuations are minor (typically within several minutes) compared to the orders-of-magnitude difference in the ablation study (RQ2), confirming that ARIADNE is not critically sensitive to these weights.

Graph Fusion Factor: Similarly, we analyzed the fusion factor α , which balances data-flow ($\alpha = 1$) and state-change ($\alpha = 0$) signals. As shown in Fig. 10, performance is optimal when both signals are integrated (e.g., $\alpha \in [0.2, 0.8]$), while relying solely on one signal consistently yields suboptimal results. This validates our design of fusing complementary dependency signals. The performance variation within the optimal range is small, further underscoring ARIADNE’s stability.

Different-Type Mismatch Penalty Sensitivity: We further evaluate sensitivity to the mismatch penalty hyperparameter ($C_{mismatch}$), which controls the tolerance for mismatch during candidate matching. Fig. 11 shows that ARIADNE remains robust across all tested values (0, 0.5, 1, 2, 5): vulnerabilities are still found in all three representative benchmarks, with moderate TTV variation. Aggregating across benchmarks, $C_{mismatch} = 2$ yields the lowest macro-average TTV, while 0.5 and 5 remain competitive. This indicates that ARIADNE is not brittle to this hyperparameter. Given this insensitivity, we set $C_{mismatch} = 2$ as the default configuration across all other experiments.

Other Parameters: We also found that other standard hyperparameters, such as the decay rate for the transition heuristic and the seed scoring weights, had a minor impact on performance.

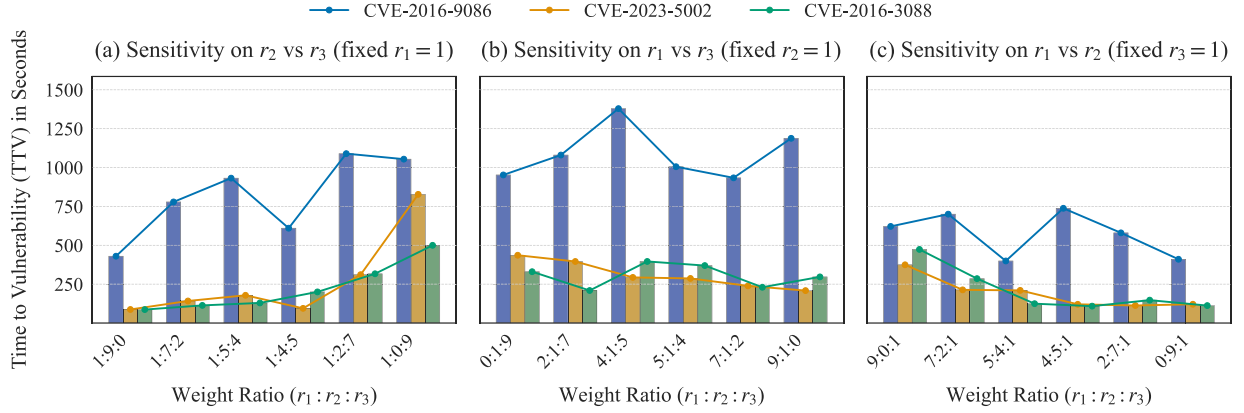


Fig. 9. Sensitivity analysis of heuristic weights (r_1, r_2, r_3) on TTV across three benchmarks. Each subplot shows the TTV as the ratio between two weights is varied while the third is held constant, demonstrating the relative importance of each heuristic.

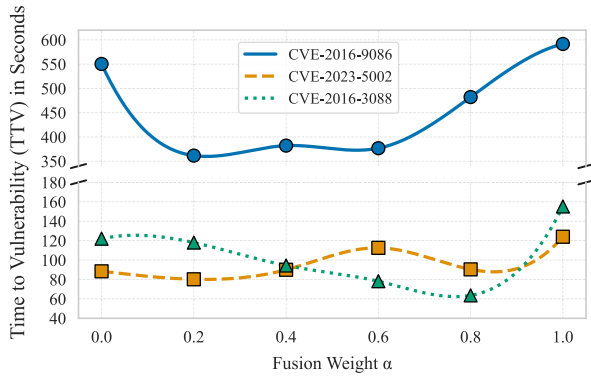


Fig. 10. Impact of the fusion hyperparameter α on TTV. α balances the weight between data flow (P_{flow}) and state change (P_{state}) probabilities. Results show that a blended approach ($\alpha \in (0, 1)$) consistently outperforms relying on a single signal.

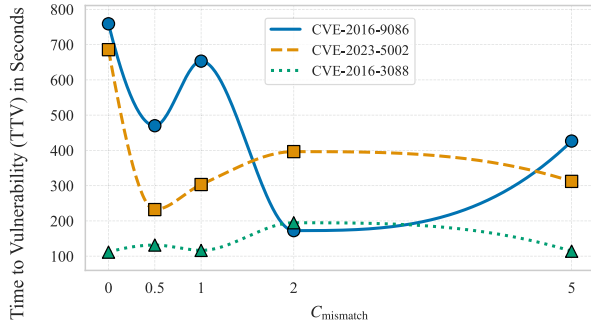


Fig. 11. Sensitivity of $C_{mismatch}$ on TTV across three representative benchmarks. Each point shows the mean TTV over successful runs. Lower TTV is generally achieved at moderate values (especially around $C_{mismatch} = 2$), while extreme settings are less stable across benchmarks.

We thus used standard default values for them throughout our evaluation.

VII. LIMITATION AND DISCUSSION

Limited Awareness of Opaque Session State: ARIADNE's state model primarily tracks persistent backend data, such as database

records and files. While many session-based states are architecturally mapped to backend stores (e.g., Redis, memory-based session tables), the current prototype does not natively decode opaque session tokens or cookies to establish a semantic link between a client-side identifier and the corresponding server-side session record. As a result, when exploring authentication-related state transitions, the search may rely less on precise state distance guidance and more on heuristic trial-and-error. From an engineering perspective, this can be mitigated by incorporating a session-decoding module: security practitioners can provide lightweight mapping scripts that resolve tokens to their backend records, which are then treated as additional tables in the state portrait and enable more precise distance computation.

Dependence on Representative Normal Traffic: ARIADNE builds its data-driven guidance models from logs of normal application usage. The quality of the API Dependency Graph and the Parameter-Source Correlation Model therefore relies on the availability of traffic that captures the typical logic exercised by the involved APIs. In production environments, such data is typically accessible through API gateways, staging logs, or existing QA test suites. For applications with sparse traffic, structured test suites or automated crawlers can generate an initial baseline. Because the models are probabilistic, they do not require exhaustive coverage but benefit from a representative sample of normal application behavior.

Onboarding Effort for New Applications: Deploying ARIADNE on a new web application involves a one-time setup cost that consists of two main parts: (i) modeling the application's state to build adapters that normalize diverse storage formats into a unified table-based representation (averaging 58 lines of code in our experiments); and (ii) collecting representative traffic to construct the offline guidance models. This initial investment is reusable across all future security incidents involving the same application. The only vulnerability-specific task is providing a success oracle, which is standard practice in forensic analysis. While this separation of concerns makes ARIADNE practical for repeated use, the upfront effort may still reduce immediate scalability in highly heterogeneous enterprise environments. Future work could explore automating parts of the adapter construction and traffic acquisition pipeline.

Support for Diverse Persistence Layers: The current prototype focuses on SQL databases and local file systems, which cover the vast majority of reported web vulnerabilities. Modern applications, however, may also rely on NoSQL databases, distributed caches (e.g., Redis), or message queues (e.g., Kafka). ARIADNE's modular state abstraction layer is designed to allow rapid integration of such components through lightweight parsing plugins that map their data models into the framework's table-based representation. Extending the plugin ecosystem would broaden the tool's coverage to match the diversity of real-world tech stacks.

Extending to Fileless Attacks: ARIADNE currently targets attacks that modify persistent backend state. Attacks that reside purely in memory (e.g., memory-only webshells, in-memory payload injections) do not leave persistent artifacts and therefore fall outside the current reconstruction scope. A promising direction for future work is to incorporate memory forensics capabilities that capture relevant in-memory data structures as part of the target state portrait. Once such memory entities are abstracted into the same table-based formalism, the existing bipartite distance framework could be reapplied to guide the reconstruction of fileless attack chains.

VIII. CONCLUSION

In this paper, we introduced ARIADNE, a novel framework for automatically reconstructing the prerequisite sequences for stateful web attacks. By formalizing the attack reproduction problem as a goal-oriented state search task, ARIADNE systematically addresses the limitations of traditional methods. Its core contributions include a method for constructing a target state portrait from syscall analysis, a state distance navigation system based on minimum weight bipartite matching, and a sequence generation engine powered by a data-driven API dependency graph and a state-driven parameter sourcing mechanism. These techniques enable ARIADNE to reason backward from the effect of the final attack to efficiently discover its cause. Our evaluation on a benchmark of 15 real-world vulnerabilities demonstrates that ARIADNE significantly outperforms existing approaches in both success rate and efficiency, reliably reconstructing complex attack chains of up to 8 steps. ARIADNE provides an effective solution to the critical security need for automated attack reproduction and opens a new research direction for state-aware web security analysis.

REFERENCES

- [1] "2024 state of the api report," 2024. [Online]. Available: <https://www.postman.com/state-of-api/2024/>
- [2] "OWASP API security project," 2026. [Online]. Available: <https://owasp.org/www-project-api-security/>
- [3] W. Li and Y. Guo, "Poster: Automated dependency mapping for web api security testing using large language models," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 5024–5026.
- [4] G. Deng et al., "NAUTILUS: Automated RESTful API vulnerability detection," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 5593–5609.
- [5] "How to test a rest API," 2026. [Online]. Available: <https://docs.rapid7.com/appspider/how-to-test-a-rest-api/>
- [6] R. Wang, Y. Peng, Y. Sun, X. Zhang, H. Wan, and X. Zhao, "TeSec: Accurate server-side attack investigation for web applications," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2023, pp. 2799–2816.
- [7] K. H. Lee, X. Zhang, and D. Xu, "High accuracy attack provenance via binary-based execution partition," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2013, pp. 1–16.
- [8] L. Yu et al., "ALchemist: Fusing application and audit logs for precise attack provenance without instrumentation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–18.
- [9] S. Ma, J. Zhai, F. Wang, K. H. Lee, X. Zhang, and D. Xu, "MPI: Multiple perspective attack investigation with semantic aware execution partitioning," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 1111–1128.
- [10] M. Alhanahnah et al., "autoMPI: Automated multiple perspective attack investigation with semantics aware execution partitioning," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2761–2775, Apr. 2023.
- [11] W. U. Hassan, M. A. Noureddine, P. Datta, and A. Bates, "OmegaLog: High-fidelity attack investigation via transparent multi-layer log analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–16.
- [12] Z. Cheng et al., "Kairos: Practical intrusion detection and investigation using whole-system provenance," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2024, pp. 3533–3551.
- [13] J. Zengy et al., "SHADEWATCHER: Recommendation-guided cyber threat analysis using system audit records," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2022, pp. 489–506.
- [14] S. Wang et al., "THREATTRACE: Detecting and tracing host-based threats in node level through provenance graph learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3972–3987, 2022.
- [15] Q. Wang et al., "You are what you do: Hunting stealthy malware via data provenance analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–17.
- [16] A. Goyal, G. Wang, and A. Bates, "R-CAID: Embedding root cause analysis within provenance-based intrusion detection," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2024, pp. 3515–3532.
- [17] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, doi: [10.14722/ndss.2020.24046](https://doi.org/10.14722/ndss.2020.24046).
- [18] C. Xiong et al., "Conan: A practical real-time APT detection system with high accuracy and efficiency," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 551–565, Jan./Feb. 2022.
- [19] X. Han et al., "SIGL: Securing software installations through deep graph learning," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2345–2362.
- [20] "Php webapps exploit," 2019. [Online]. Available: <https://www.exploit-db.com/exploits/46635>
- [21] "Cms made simple serverside template injection remote code execution vulnerability," 2021. [Online]. Available: <https://srcincite.io/pocs/cve-2021-26120.py.txt>
- [22] E. Tricket et al., "Toss a fault to your Witcher: Applying grey-box coverage-guided mutational fuzzing to detect SQL and command injection vulnerabilities," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2023, pp. 2658–2675.
- [23] E. Güler et al., "ATROPOS: Effective fuzzing of web applications for server-side vulnerabilities," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 4765–4782.
- [24] Z. Lin et al., "Effective directed fuzzing with hierarchical scheduling for web vulnerability detection," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 8349–8366.
- [25] C. Wang, W. Meng, C. Luo, and P. Li, "Predator: Directed web application fuzzing for efficient vulnerability validation," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2025, pp. 886–902.
- [26] "Representational state transfer," 2026. [Online]. Available: https://en.wikipedia.org/wiki/Representational_state_transfer
- [27] C. Lyu et al., "Miner: A hybrid data-driven approach for restapi fuzzing," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 4517–4534.
- [28] W. Du et al., "Vulnerability-oriented testing for RESTful APIs," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 739–755.
- [29] H. W. Kuhn, "The Hungarian method for the assignment problem," *Nav. Res. Logist. Quart.*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [30] R. E. Tarjan, *Data Structures Netw. Algorithms*. Philadelphia, PA, USA: SIAM, 1983.
- [31] A. Bates, D. J. Tian, K. R. Butler, and T. Moyer, "Trustworthy whole-system provenance for the linux kernel," in *Proc. 24th USENIX Secur. Symp.*, 2015, pp. 319–334.
- [32] T. Pasquier et al., "Practical whole-system provenance capture," in *Proc. Symp. Cloud Comput.*, 2017, pp. 405–418.
- [33] "Apache solr," 2026. [Online]. Available: <https://solr.apache.org/>
- [34] "The most-comprehensive AI-powered devsecops platform," 2026. [Online]. Available: <https://about.gitlab.com/>
- [35] "pgAdmin - postgresSQL tools," 2026. [Online]. Available: <https://www.pgadmin.org/>

- [36] "ActiveMQ," 2026. [Online]. Available: <https://activemq.apache.org/>
- [37] "Apache airflow," 2026. [Online]. Available: <https://airflow.apache.org/>
- [38] "Open source content management system | cms made simple," 2026. [Online]. Available: <https://www.cmsmadesimple.org/>
- [39] "Elasticsearch: The official distributed search & analytics engine| Elastic," 2026. [Online]. Available: <https://www.elastic.co/elasticsearch/>
- [40] "Apache couchDB," 2026. [Online]. Available: <https://couchdb.apache.org/>
- [41] "DedeCMS official website," 2025. [Online]. Available: <https://www.dedecms.com/>
- [42] "Yapi," 2026. [Online]. Available: <https://github.com/YMFE/yapi/>
- [43] M. Levandowsky and D. Winter, "Distance between sets," *Nature*, vol. 234, no. 5323, pp. 34–35, 1971.
- [44] Y. Peng, B. Ma, H. Wan, and X. Zhao, "Ancora: Accurate intrusion recovery for web applications," 2025, arXiv:2510.07806.
- [45] A. Bates et al., "Transparent web service auditing via network provenance functions," in *Proc. 26th Int. Conf. World Wide Web*, 2017, pp. 887–895.
- [46] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 2329–2344.
- [47] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as Markov chain," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 1032–1043.
- [48] "Open-source collection of pre-built vulnerable docker environments for security researchers and educators," (n.d.). [Online]. Available: <https://vulhub.org/>
- [49] "Kanboard," 2026. [Online]. Available: <https://github.com/kanboard/kanboard/>
- [50] "Admidio," 2026. [Online]. Available: <https://github.com/Admidio/admidio>
- [51] "Yapi registration and mock remote code execution," 2026. [Online]. Available: <https://github.com/vulhub/vulhub/tree/master>
- [52] "Yapi RCE vulnerability reproduce," 2021. [Online]. Available: <https://www.freebuf.com/vuls/279967.html>



Yihao Peng (Graduate Student Member, IEEE) received the BEng degree from the School of Software, Tsinghua University in 2023. He is currently working toward the PhD degree in Software Engineering with Tsinghua University. His research interests include system security, web security, and AI for security.



Jingrun Zhang received the BEng degree in computer science and technology from Northwestern Polytechnical University in 2024. He is currently working toward the MS degree in software engineering with Tsinghua University. His research interests include system security and web security.



Xinyu Mu received the BEng degree in computer science and technology from Beijing Institute of Technology in 2024. He is currently working toward the MS degree in software engineering with Tsinghua University. His research interests include web security and automated web vulnerability mining.



Zhuofeng He received the MSc degree in computer science from Jinan University in 2024. He is currently working toward the PhD degree in electronic information with Central South University. His research interests include web security and automated web vulnerability mining.



Shouling Ji (Senior Member, IEEE) received the first PhD degree in electrical and computer engineering from Georgia Institute of Technology, and the second PhD degree in computer science from Georgia State University. He is currently a full professor with the College of Computer Science and Technology, Zhejiang University. He is also a research faculty with the School of Electrical and Computer Engineering, Georgia Institute of Technology (Georgia Tech). His research interests include data-driven security and privacy, AI security, and Big Data analytics.



Hai Wan received the BS degree in computer science and technology from the National University of Defense Technology, Changsha, China, in 2003, and the PhD degree in computer science and technology from Tsinghua University, Beijing, China, in 2011. From 2011 to 2012, he was a postdoc researcher with INRIA, Nancy, France. From 2013, he is with the Faculty of Tsinghua University, Beijing, China. He is currently an associate professor with the School of Software, Tsinghua University. His research focuses on software security.



Xibin Zhao (Senior Member, IEEE) received the BS, ME, and PhD degrees from the School of Computer Science and Telecommunication Engineering from Jiangsu University, in 1994, 2000, and 2004, respectively. He is currently an associate professor with the School of Software, Tsinghua University. His research interests include industrial cybersecurity, software security, and artificial intelligence security.